

Phase 1: Data Cleaning & Quality Check

StreamFlix Content Analytics Project

Name: Mark Maxwell Louis

In this notebook I load the 6 CSV files and check the data for problems before doing any real analysis. I go table by table and check the things listed in the project brief: missing values, duplicate IDs, whether dates make sense, and a few other checks.

In [1]:

```
import pandas as pd

pd.set_option('display.max_columns', None)
```

Step 1: Load the 6 files

In [2]:

```
subscribers = pd.read_csv('subscribers.csv')
titles = pd.read_csv('titles.csv')
watch_history = pd.read_csv('watch_history.csv')
ratings = pd.read_csv('ratings.csv')
reviews = pd.read_csv('reviews.csv')
watchlist = pd.read_csv('watchlist.csv')

print('subscribers:', subscribers.shape)
print('titles:', titles.shape)
print('watch_history:', watch_history.shape)
print('ratings:', ratings.shape)
print('reviews:', reviews.shape)
print('watchlist:', watchlist.shape)
```

```
subscribers: (15000, 14)
titles: (9000, 21)
watch_history: (650000, 10)
ratings: (130000, 5)
reviews: (110000, 7)
watchlist: (65000, 5)
```

Step 2: Convert date columns

When you read a CSV with `pd.read_csv`, dates come in as plain text. I need to convert them to real dates with `pd.to_datetime` so I can compare and subtract them later.

In [3]:

```
subscribers['signup_date'] = pd.to_datetime(subscribers['signup_date'])
subscribers['churn_date'] = pd.to_datetime(subscribers['churn_date'])

titles['date_added'] = pd.to_datetime(titles['date_added'])
titles['license_expiry'] = pd.to_datetime(titles['license_expiry'])

watch_history['watch_date'] = pd.to_datetime(watch_history['watch_date'])
ratings['rating_date'] = pd.to_datetime(ratings['rating_date'])
reviews['review_date'] = pd.to_datetime(reviews['review_date'])
watchlist['added_date'] = pd.to_datetime(watchlist['added_date'])

print(subscribers.dtypes)
```

```
subscriber_id      str
signup_date        datetime64[us]
country            str
region            str
age               int64
gender            str
plan_type         str
monthly_price_usd  float64
household_size     int64
primary_device     str
payment_method     str
tenure_months      int64
is_active          bool
churn_date         datetime64[us]
dtype: object
```

Step 3: Check for missing values

I'll check each table one at a time with `.isna().sum()` .

In [4]:

```
print('--- subscribers ---')
print(subscribers.isna().sum())
```

```
--- subscribers ---
subscriber_id      0
signup_date        0
country            0
region            0
age               0
gender            0
plan_type         0
monthly_price_usd  0
household_size     0
primary_device     0
payment_method     0
tenure_months      0
is_active          0
churn_date        11199
dtype: int64
```

In [5]:

```
print('--- titles ---')
print(titles.isna().sum())
```

```
--- titles ---
title_id          0
title_name        0
type              0
primary_genre     0
country           0
language          0
release_year      0
date_added        0
maturity_rating   0
seasons           0
content_duration_min 0
is_original       0
license_type      0
director          0
cast              0
quality_score     0
popularity_score  0
license_cost_usd  0
license_expiry    1966
total_watch_hours 0
total_plays       0
dtype: int64
```

In [6]:

```
print('--- watch_history ---')
print(watch_history.isna().sum())
```

```
--- watch_history ---
watch_id          0
subscriber_id     0
title_id          0
watch_date        0
device            0
region            0
content_duration_min 0
watch_duration_min 0
completion_pct    0
completed         0
dtype: int64
```

In [7]:

```
print('--- ratings ---')
print(ratings.isna().sum())
```

```
--- ratings ---
rating_id         0
subscriber_id     0
title_id          0
rating            0
rating_date       0
dtype: int64
```

In [8]:

```
print('--- reviews ---')
print(reviews.isna().sum())
```

```
--- reviews ---
review_id      0
subscriber_id  0
title_id       0
review_text    0
sentiment      0
helpful_votes  0
review_date    0
dtype: int64
```

In [9]:

```
print('--- watchlist ---')
print(watchlist.isna().sum())
```

```
--- watchlist ---
watchlist_id    0
subscriber_id    0
title_id         0
added_date      0
watched          0
dtype: int64
```

What I found: The only columns with missing values are `churn_date` in subscribers and `license_expiry` in titles. Both of these make sense:

- `churn_date` is blank when a subscriber has never churned (still active).
- `license_expiry` is blank for StreamFlix Originals, because the company owns them outright and they don't have an expiry date.

So these aren't really "missing data" problems, they are expected blanks.

Step 4: Check for duplicate IDs

Each table has a primary key that should be unique. I'll check with `.duplicated().sum()`.

In [10]:

```
print('duplicate subscriber_id:', subscribers['subscriber_id'].duplicated().sum())
print('duplicate title_id:', titles['title_id'].duplicated().sum())
print('duplicate watch_id:', watch_history['watch_id'].duplicated().sum())
print('duplicate rating_id:', ratings['rating_id'].duplicated().sum())
print('duplicate review_id:', reviews['review_id'].duplicated().sum())
print('duplicate watchlist_id:', watchlist['watchlist_id'].duplicated().sum())
```

```
duplicate subscriber_id: 0
duplicate title_id: 0
duplicate watch_id: 0
duplicate rating_id: 0
duplicate review_id: 0
duplicate watchlist_id: 0
```

Result: No duplicates in any table. All the IDs are unique.

Step 5: Check churn logic

The brief asks me to check two things:

1. For churned subscribers, is `churn_date` after `signup_date`?
2. Do active subscribers have a blank `churn_date`?

In [11]:

```
churned = subscribers[subscribers['is_active'] == False]
active = subscribers[subscribers['is_active'] == True]

# Check 1: churn_date should be after signup_date
bad_dates = churned[churned['churn_date'] < churned['signup_date']]
print('Churned subscribers where churn_date is before signup_date:', len(bad_dates))
```

Churned subscribers where churn_date is before signup_date: 0

In [12]:

```
# Check 2: active subscribers should have an empty churn_date
active_with_churn_date = active[active['churn_date'].notna()]
print('Active subscribers that still have a churn_date filled in:', len(active_with_churn_date))

# Also check the opposite: churned subscribers missing a churn_date
churned_missing_date = churned[churned['churn_date'].isna()]
print('Churned subscribers missing a churn_date:', len(churned_missing_date))
```

Active subscribers that still have a churn_date filled in: 0

Churned subscribers missing a churn_date: 0

What I found: All three checks came back with 0 rows. The churn dates are consistent.

Step 6: Check completion_pct

The brief says `completion_pct` should roughly equal `watch_duration_min / content_duration_min * 100`. I'll calculate that myself and compare it to the value already in the table.

In [13]:

```
watch_history['completion_check'] = watch_history['watch_duration_min'] /
watch_history['content_duration_min'] * 100
watch_history['completion_difference'] = watch_history['completion_check'] -
watch_history['completion_pct']
watch_history['completion_difference'] = watch_history['completion_difference'].abs()

print(watch_history['completion_difference'].describe())
```

```
count      650000.000000
mean         0.029353
std         0.021061
min         0.000000
25%         0.012500
50%         0.026531
75%         0.042373
max         0.155556
Name: completion_difference, dtype: float64
```

In [14]:

```
large_mismatch = watch_history[watch_history['completion_difference'] > 5]
print('Rows where the difference is more than 5 percentage points:', len(large_mismatch))
print('Out of total rows:', len(watch_history))
```

Rows where the difference is more than 5 percentage points: 0

Out of total rows: 650000

Result: The biggest difference across all 650,000 rows is under 1 percentage point, which is just rounding. `completion_pct` matches the calculation.

Step 7: Check sentiment values in reviews

The brief says `sentiment` should only be Positive, Neutral, or Negative.

In [15]:

```
print(reviews['sentiment'].value_counts())
```

```
sentiment
Positive    75007
Neutral     22759
Negative    12234
Name: count, dtype: int64
```

In [16]:

```
ok_values = ['Positive', 'Neutral', 'Negative']
bad_sentiment = reviews[~reviews['sentiment'].isin(ok_values)]
print('Rows with an unexpected sentiment value:', len(bad_sentiment))
```

Rows with an unexpected sentiment value: 0

Conclusion: Only the 3 expected values appear. No bad data here.

Step 8: Check watchlist references

Does every row in watchlist.csv point to a real subscriber and a real title?

In [17]:

```
watchlist_bad_subscriber = watchlist[~watchlist['subscriber_id'].isin(subscribers['subscriber_id'])]
watchlist_bad_title = watchlist[~watchlist['title_id'].isin(titles['title_id'])]

print('Watchlist rows with a subscriber_id that does not exist:', len(watchlist_bad_subscriber))
print('Watchlist rows with a title_id that does not exist:', len(watchlist_bad_title))
```

Watchlist rows with a subscriber_id that does not exist: 0
Watchlist rows with a title_id that does not exist: 0

I did the same check for watch_history, ratings, and reviews too, since they all reference subscribers and titles the same way.

In [18]:

```
print('watch_history bad subscriber_id:',
      (~watch_history['subscriber_id'].isin(subscribers['subscriber_id'])).sum())
print('watch_history bad title_id:', (~watch_history['title_id'].isin(titles['title_id'])).sum())
print('ratings bad subscriber_id:', (~ratings['subscriber_id'].isin(subscribers['subscriber_id'])).sum())
print('ratings bad title_id:', (~ratings['title_id'].isin(titles['title_id'])).sum())
print('reviews bad subscriber_id:', (~reviews['subscriber_id'].isin(subscribers['subscriber_id'])).sum())
print('reviews bad title_id:', (~reviews['title_id'].isin(titles['title_id'])).sum())
```

watch_history bad subscriber_id: 0

watch_history bad title_id: 0

ratings bad subscriber_id: 0
ratings bad title_id: 0
reviews bad subscriber_id: 0
reviews bad title_id: 0

Result: 0 bad references everywhere. Every row links to a real subscriber and a real title.

Step 9: Check for impossible watch durations

Can a session be longer than the actual content? That shouldn't be possible.

In [19]:

```
too_long = watch_history[watch_history['watch_duration_min'] > watch_history['content_duration_min']]
print('Sessions longer than the content itself:', len(too_long))
print('Percent of all sessions:', round(len(too_long) / len(watch_history) * 100, 2), '%')
```

Sessions longer than the content itself: 0
Percent of all sessions: 0.0 %

Result: 0 sessions. No impossible durations.

Step 10: Subscribers with short tenure but marked active

The brief says this is just something to note, not necessarily an error.

In [20]:

```
short_tenure_active = subscribers[(subscribers['is_active'] == True) & (subscribers['tenure_months'] < 2)]
print('Active subscribers with less than 2 months tenure:', len(short_tenure_active))
print('Percent of all subscribers:', round(len(short_tenure_active) / len(subscribers) * 100, 2), '%')
```

Active subscribers with less than 2 months tenure: 321
Percent of all subscribers: 2.14 %

What I found: 321 subscribers (about 2%) are recent sign-ups. This makes sense since new people join every month, so it is not a data problem.

Summary of my data quality checks

Check		Result
Missing values	Only churn_date and license_expiry are blank, and that is expected	
Duplicate IDs		0 in every table
Churn date logic		0 problems
completion_pct matches the formula		Yes, within rounding
sentiment values	Only Positive/Neutral/Negative	
Watchlist/watch_history/ratings/reviews references		0 bad references
Sessions longer than content		0
Active subs with short tenure		321, expected

Conclusion: The data is clean. I did not find anything that needs to be fixed before moving on to Phase 2 and Phase 3.

In [21]:

```
# Save the cleaned tables (with real date types) so I can reuse them in Phase 2 and Phase 3
subscribers.to_csv('subscribers_clean.csv', index=False)
titles.to_csv('titles_clean.csv', index=False)
watch_history.drop(columns=['completion_check', 'completion_difference']).to_csv('watch_history_clean.csv',
index=False)
ratings.to_csv('ratings_clean.csv', index=False)
reviews.to_csv('reviews_clean.csv', index=False)
watchlist.to_csv('watchlist_clean.csv', index=False)
print('Saved cleaned files.')
```

Saved cleaned files.